

Neural networks, machine learning and randomness

Stochastic methods for artificial neural networks

Optional course for master students

Martin Holeňa



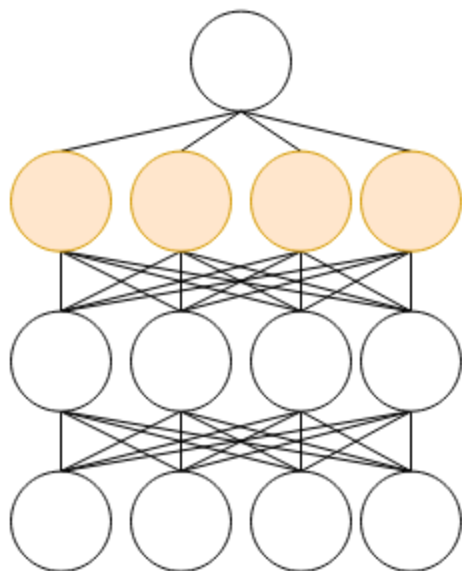
What will it be about?

- ◆ Dropout and its role in neural network learning
 - original Bernoulli dropout in multilayer perceptrons
 - dropout in other kinds of models
 - Gaussian dropout, its relation to Bernoulli dropout
- ◆ Stochastic gradient descent and its assumptions
 - stochastic gradient descent viewed as a random process

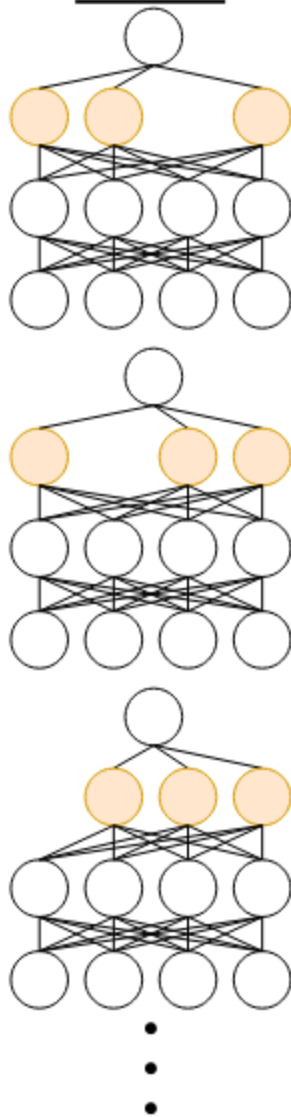
Dropout

- ◆ Temporarily *removing* (dropping out) *some* input or hidden *neurons during* network *training*
- ◆ Neurons are dropped out *randomly*, according to a given distribution
- ◆ Originally proposed for and most often used during training of multilayer perceptrons

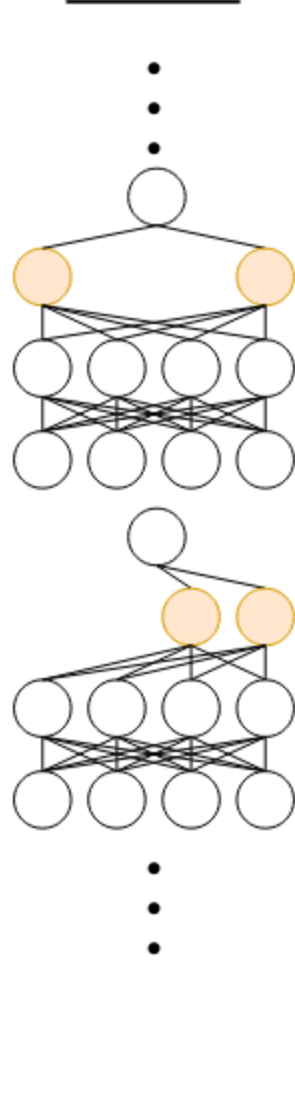
Original network



One dropped neuron

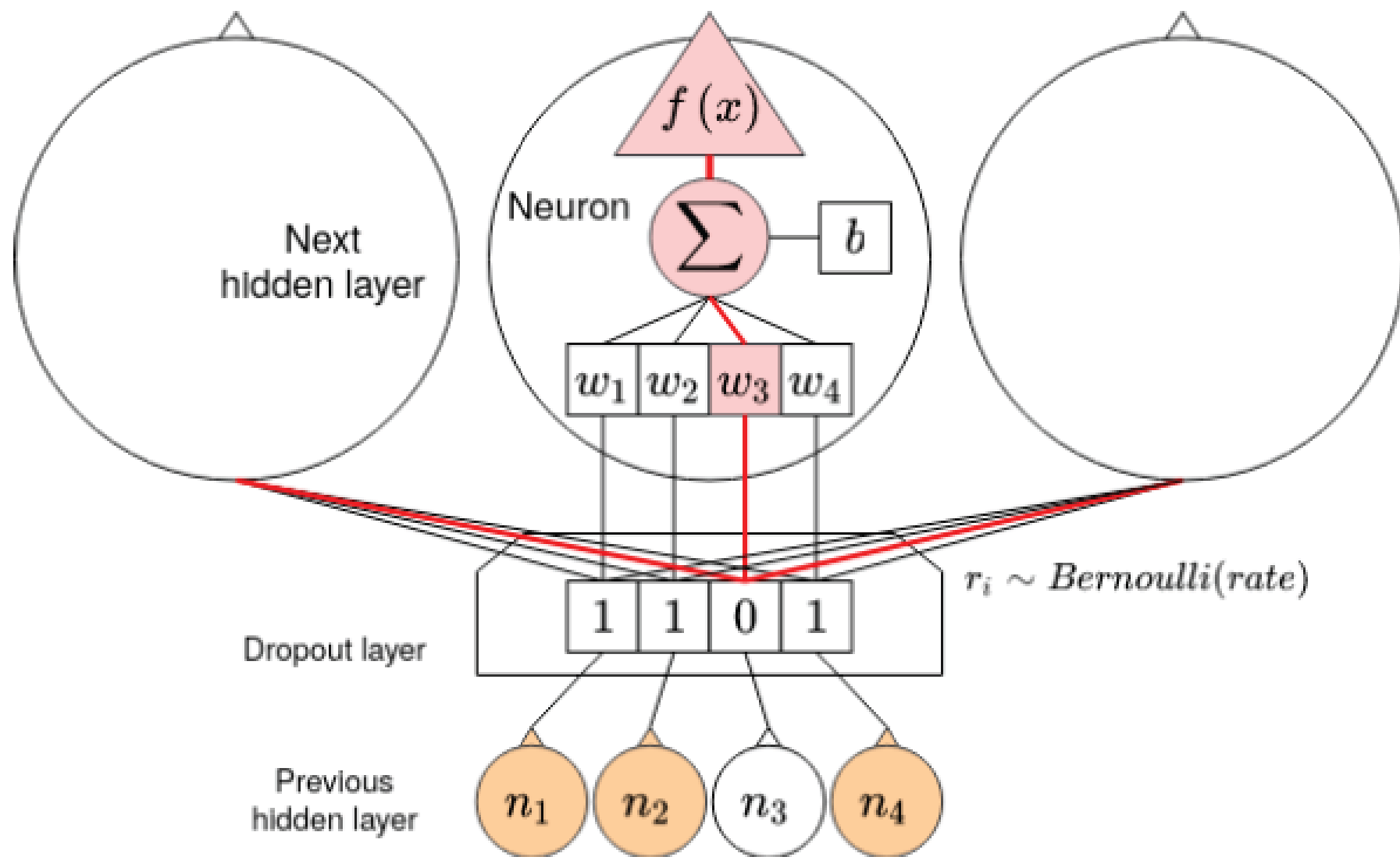


Two dropped neurons



Bernoulli dropout

- ◆ Bernoulli(p) distribution: on $\{a, b\}$ with probabilities $(1 - p, p)$
- ◆ Assumptions about l -th *hidden layer*, $l = 1, \dots, L$:
 - vectorial input $z^{(l)}$, output $y^{(l)}$, weight $w^{(l)}$, scalar bias $b^{(l)}$
 - activation function f does not depend on l , relates $y^{(l)} = f(z^{(l)})$
 - in addition: set $a = 0, b = 1$, denote $y^{(0)} = x$ – network input
- ◆ Then $z_i^{(l)} = w_i^{(l)} r_i^{(l)} y_i^{(l-1)} + b^{(l)}$, with *random* $r_i^{(l)} \sim \text{Bernoulli}(p)$



Dropout and network training

- ◆ Most often using stochastic gradient descent
- ◆ Difference from standard MLP: for each *training case*,
new values $r_i^{(l)}$ are sampled $\Rightarrow$ a new *specific network*
 - forward- and backpropagation restricted to that individual network
- ◆ *Gradients* are *averaged* over cases retaining the parameter
 - cases with that parameter dopped out $\Rightarrow$ gradient contribution = 0

Dropout and regularization

- ◆ Dropout alone improves training, with regularization even more
- ◆ Most often combined with *max-norm regularization*: $\|w\| \leq c$
 - w – vector of all weights, $\| \quad \|$ – some norm, c – hyperparameter
 - $\Rightarrow$ network learning is then constrained optimization
- ◆ Main reason why max-norm regularization is useful:
no weights blowup through large learning rate $\Rightarrow$ explorability

Some other properties of dropout

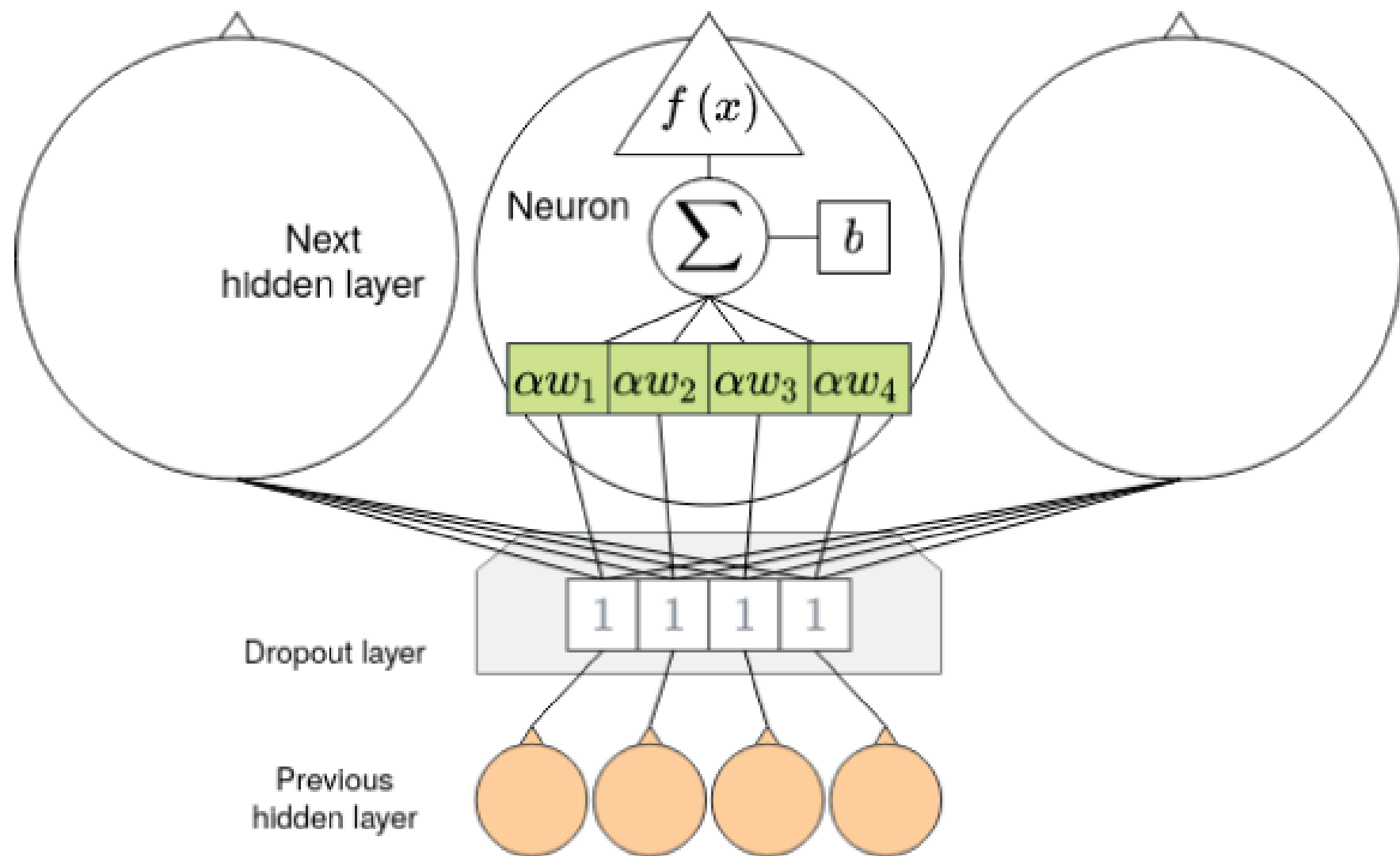
- ◆ *Sparse representation*, even if no sparsity inducing regularizers
- ◆ *Influence of dataset size* relatively to network size:
 - very small datasets overfitting even after dropout $\Rightarrow$ useless
 - with increasing dataset size, its usefulness increases, then again decreases $\Leftarrow$ for very large datasets, no overfitting occurs
- ◆ Training time: $2 - 3 \times$ longer than without dropout

Advantages of dropout

1. After dropout, the network has less parameters $\Rightarrow$
 $\Rightarrow$ less prone to *overfitting* the training data
2. Breaking-up co-adaptations of different hidden neurons,
which impede generalization $\Rightarrow$ improved *generalization*
3. Different dropout realizations $\approx$ different network topologies $\Rightarrow$
 $\Rightarrow$ dropout implies building network *ensembles*

Dropout ensembles

- ◆ For an ensemble S built through dropping out subsets of the set H of hidden neurons: $|\mathcal{S}| \leq 2^{|H|}$
- ◆ If during training, $h \in H$ survives dropout with probability p , then during testing, weights outgoing from h are multiplied by p
 - $\Rightarrow$ *expected weights after training = used testing weights*
- ◆ Alternative possibility: training weights multiplied by $\frac{1}{p}$



More general dropouts

- ◆ Used also with other models than multilayer perceptrons
 - *restricted Boltzmann* machine
 - *linear regression* (will be described later)
- ◆ Used also with other distributions than Bernoulli
 - *Gaussian distribution* (will be described later)

Dropout in linear regression

- ◆ Dropped out are individual training pairs – rows of (X, y)
 - $X \in \mathbb{R}^{N \times d}$ – matrix of N data points, $y \in \mathbb{R}^N$ – vector of targets
- ◆ Dropout introduced through a component-wise product $X \odot R$
 - $R \in \{0,1\}^{N \times d}$ is a $\{0,1\}^{N \times d}$ -valued random matrix
 - R has all its *components random* $R_{ij} \sim \text{Bernoulli}(p)$

Learning dropout in linear regression

- ◆ Learning in traditional linear regression consists in finding a weight vector $w \in \mathbb{R}^d$ minimizing the error $\|y - Xw\|^2$
- ◆ For dropout linear regression learning, the *minimized error* turns to $\mathbb{E}_{R \sim \text{Bernoulli}(p)} \|y - X \odot R w\|^2 =$ (after computation)
$$= \|y - pXw\|^2 + p(1-p) \left\| \left(\text{diag}(X^\top X) \right)^{\frac{1}{2}} w \right\|^2 =$$
$$= \|y - X\tilde{w}\|^2 + \frac{1-p}{p} \left\| \left(\text{diag}(X^\top X) \right)^{\frac{1}{2}} \tilde{w} \right\|^2, \text{ with } \tilde{w} = pw$$

Gaussian dropout

- ◆ Basic idea: *activation* h_i of the hidden neuron i is perturbed to $h_i(1 + r)$ with $r \sim N(0,1)$, more generally $r \sim N(0, \sigma^2)$
- ◆ Equivalently: activation h_i is *perturbed to* $h_i r'$ with $r' = 1 + r$, hence $r' \sim N(1,1)$, more generally $r' \sim N(1, \sigma^2)$
- ◆ *Hyperparameter* σ^2 , like p in Bernoulli dropout

What does the Gaussian drop out?

- ◆ Formally, Gaussian dropout drops no neurons out, only perturbs the activations of hidden neurons
- ◆ However, for $h_i r'$ with $r' \sim N(1, \sigma^2)$, where $\sigma^2 = \frac{1-p}{p}$:
the expectation and variance of r' are $\mathbb{E} r' = 1, \text{Var } r' = \frac{1-p}{p}$
- ◆ And the same $\mathbb{E} r'$ and $\text{Var } r'$ has $r' \sim \text{Bernoulli}(p)$ on $\left\{0, \frac{1}{p}\right\}$,
which drops out the hidden neuron i

Stochastic gradient

- ◆ Minimizing a loss function for data $z = (z_1, \dots, z_N)$, parameters θ , summed over data: $\mathcal{L}(\theta) = \mathcal{L}(\theta, z) = \frac{1}{N} \sum_{n \in \hat{N}} \ell_n(\theta, z_n)$, $\hat{N} = \{1, \dots, N\}$
- ◆ For $s \in \mathbb{N}$, consider a random variable $\mathcal{M}_s: \{S \subset \hat{N} | \#S = s\}$ -valued, called minibatch, uniformly distributed: $S \subset \hat{N} \ \& \ \#S = s \Rightarrow P(S) = \frac{1}{\binom{N}{s}}$
- ◆ Define a random loss function: $\hat{\mathcal{L}}_s(\theta) = \hat{\mathcal{L}}_s(\theta, z) = \frac{1}{s} \sum_{n \in \mathcal{M}_s} \ell_n(\theta, z_n)$
- ◆ Its *gradient* $\hat{g}_s = \nabla_{\theta} \hat{\mathcal{L}}_s$ is called *stochastic gradient*

Example: quadratic loss

- ◆ $\ell_n(\theta, z_n) = \frac{1}{2} \|z_n - \theta\|^2, \mathcal{L}(\theta) = \mathcal{L}(\theta, z) = \frac{1}{2N} \sum_{n=1}^N \|z_n - \theta\|^2$
- ◆ $\nabla_{\theta} \mathcal{L}(\theta, z) = \bar{z} - \theta$, with $\bar{z} = \sum_{n=1}^N z_n$, thus $\arg \min_{\theta} \mathcal{L}(\theta, z) = \bar{z}$
 - the reparametrization $\theta_{\text{new}} = \theta - \bar{z}$ leads to $\arg \min_{\theta_{\text{new}}} \mathcal{L}(\theta_{\text{new}}, z) = 0$
- ◆ $\hat{\mathcal{L}}_s(\theta) = \frac{1}{2s} \sum_{n \in \mathcal{M}} \|z_n - \theta\|^2, \hat{g}_s(\theta) = \nabla_{\theta} \hat{\mathcal{L}}_s(\theta) = \frac{1}{s} \sum_{n \in \mathcal{M}} (z_n - \theta)$

Recalling standard GD

- ◆ Standard gradient descent—also called *complete*, or *batch*

$$w^+ = w - \frac{\eta}{N} \sum_{n=1}^N \nabla_{\theta} \ell_n(\theta, z_n)$$

- *epoch*: one update in this way

- ◆ Advantage: updates independent of local differences $\Rightarrow$ stable
- ◆ Disadvantage: computationally demanding on large-scale data
 - unusable for datasets that don't fit in memory

Stochastic gradient descent (SGD)

- ◆ The application of gradient descent to stochastic gradient:

$$\theta_{t+1} = \theta_t - \eta \hat{g}_s(\theta_t, z) = \theta_t - \eta \nabla_{\theta} \hat{\mathcal{L}}_s(\theta_t, z)$$

- θ_t – parameters in the iteration t , η – learning rate
- $\hat{g}_s(\theta, z) = \nabla_{\theta} \hat{\mathcal{L}}_s(\theta, z) = \frac{1}{s} \sum_{n \in \mathcal{M}_s} \nabla_{\theta} \ell_n(\theta, z_n)$

- ◆ $s < N \Rightarrow$ SGD is often called minibatch gradient descent

SGD pseudocode

- ◆ Initialize $\eta > 0$ and $\theta_0 \in \mathbb{R}^d$ to a given value
- ◆ Perform a *while loop* until termination criteria:
 1. $\zeta_t \leftarrow \nabla_{\theta} \hat{\mathcal{L}}_s(\theta_t, z) = \frac{1}{s} \sum_{n \in \mathcal{M}} \nabla_{\theta} \ell_n(\theta_t, z_n)$
 2. $\theta_{t+1} \leftarrow \theta_t - \eta \zeta_t$
 3. $t \leftarrow t + 1$

Comparing GD and SGD

- ◆ *Advantages of GD*: stable $\Rightarrow$ converges in a small number of iterations, converges to a local minimum, for convex functions to a global minimum
- ◆ *Advantages of SGD*: fast and memory efficient
 $\Rightarrow$ is suitable for online learning

Stochastic average gradient (SAG)

- ◆ Based on expected increments of $\nabla_{\theta} \hat{\mathcal{L}}_s(\theta_t, z) = \frac{1}{s} \sum_{n \in \mathcal{M}} \nabla_{\theta} \ell_n(\theta_t, z_n)$
- ◆ Initializes $\theta_0, d = 0, \zeta_n = 0$ for $n = 1, \dots, N$, then in each iteration t samples $\mathcal{M}_s$ and for $n \in \mathcal{M}_s$ updates

$$d = d - \zeta_n + \nabla_{\theta} \ell_n(\theta_t, z_n)$$

$$\zeta_n = \nabla_{\theta} \ell_n(\theta_t, z_n)$$

- ◆ Progress to the next iteration: $\theta_{t+1} = \theta_t + \eta d$

SGD with momentum

- ◆ Decreases SGD oscillations by means of a *momentum* $\mu \hat{g}_s(\theta, z)$
- ◆ Partially preserves the previous gradient direction $\hat{g}_s$

$$\hat{g}_s(\theta_t, z) = \mu \hat{g}_s(\theta_{t-1}, z) + \alpha \nabla_{\theta} \hat{\mathcal{L}}_s(\theta_t, z), \mu > 0, \alpha > 0$$

$$\theta_{t+1} = \theta_t - \hat{g}_s(\theta_t, z)$$

- ◆ This method is also called *stochastic HB*

⇐ the effect of μ resembles heavy ball inertia

Nesterov accelerated GD

- ◆ Another method attempting to decrease SGD oscillations
- ◆ Like SGD with momentum, but $\nabla_{\theta} \hat{\mathcal{L}}_s$ is computed in the point $\nabla_{\theta} \hat{\mathcal{L}}_s(\theta_t - \hat{g}_s(\theta_{t-1}, z), z)$ *after the descent*

$$\hat{g}_s(\theta_t, z) = \mu \hat{g}_s(\theta_{t-1}, z) + \alpha \nabla_{\theta} \hat{\mathcal{L}}_s(\theta_{t-1}, z)$$

$$\theta_{t+1} = \theta_t - \hat{g}_s(\theta_t, z)$$

Learning rate

- ◆ Important: *small* $\rightarrow$ *slow convergence*, *large* $\rightarrow$ *oscillations*
- ◆ Maybe different impact on different dimensions $\Leftarrow$ respective gradient components (= partial derivatives) differ in order of magnitude
- ◆ Remedy: set different learning rates for different dimensions
 - if dimensionality high, manual setting is time consuming
 - $\Rightarrow$ methods for adaptive learning rate determination

AdaGrad (Adaptive Gradient)

- ◆ Modifies the original learning rate η dynamically

based on the *gradients* acquired in *previous iterations*

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\text{diag}(G_{t-1}) + \eta}} \hat{g}_s(\theta_{t-1}, z)$$

- $G_t \in \mathbb{R}^d$, it sums the past gradients squares; example:

$$\begin{aligned} \hat{g}_s(\theta_1, z) &= \begin{pmatrix} 1 \\ 0 \\ 2 \end{pmatrix}, \hat{g}_s(\theta_2, z) = \begin{pmatrix} 3 \\ 4 \\ 0 \end{pmatrix}, \hat{g}_s(\theta_3, z) = \begin{pmatrix} 0 \\ 5 \\ 6 \end{pmatrix} \Rightarrow G_4 = \begin{pmatrix} 1^2 + 3^2 \\ 4^2 + 5^2 \\ 3^2 + 6^2 \end{pmatrix} = \begin{pmatrix} 10 \\ 41 \\ 40 \end{pmatrix} \\ \Rightarrow \sqrt{\text{diag}(G_4) + \epsilon} &= \begin{pmatrix} \sqrt{10 + \eta} & 0 & 0 \\ 0 & \sqrt{41 + \eta} & 0 \\ 0 & 0 & \sqrt{40 + \eta} \end{pmatrix} \end{aligned}$$

RMSProp

- ◆ Lets the influence of past iterations exponentially *decay*
⇒ removes the drawback of AdaGrad that
it treats all past gradients equally
- ◆ A momentum method, but uses the *2nd-order momentum* βv_t :

$$v_t = \beta v_{t-1} + (1 - \beta) \hat{g}_s^2(\theta_t, z), \beta > 0$$

$$\theta_t = \theta_{t-1} - \alpha_{t.1} v_{t-1}^{-1/2} \hat{g}_s(\theta_{t-1}, z)$$

Adam

- ◆ *Adaptive moment estimation* - an often fast converging combination of adaptive learning rate and momentum methods
- ◆ Uses the 1. (αm_t) + 2. (βv_t) moments of the gradient ($\alpha, \beta \in (0,1)$)

$$m_t = \alpha m_{t-1} + (1 - \alpha) \hat{g}_s(\theta_t, z), v_t = \beta v_{t-1} + (1 - \beta) \hat{g}_s^2(\theta_t, z)$$

$$\hat{m}_t = \frac{m_t}{1 - \alpha^t}, \hat{v}_t = \frac{v_t}{1 - \beta^t}$$

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t, \eta > 0$$

Modifications of Adam

- ◆ Motivated by occasional problems with Adam's convergence
- ◆ *AdamW*: modifies the *weights* (θ) update with their *decay*

$$\theta_t = \theta_{t-1} - \lambda \eta \theta_{t-2} - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t, \lambda > 0$$

- ◆ *AMSGrad*: modifies the *2. moment* of the gradient

$$\bar{v}_t = \beta \bar{v}_{t-1} + (1 - \beta) \hat{g}_s^2(\theta_t, z)$$

$$v_t = \max\{v_{t-1}, \bar{v}_t\}$$

General comments to SGD

- ◆ In the SGD context, processing 1 minibatch is denoted as *iteration*, processing all available samples as *epoch*
- ◆ At the beginning of each epoch, all samples are *randomly shuffled*, and afterwards they are *split* into multiple subsequently processed minibatches
- ◆ SGD is studied using 4 generally accepted assumptions

Assumption 1

- ◆ Conditioned on $\theta, z_1, \dots, z_n$ are *conditionally independent identically distributed* and such that $\hat{g}_s(\theta, x) = \frac{1}{s} \sum_{n \in \mathcal{M}} \nabla_{\theta} \ell_n(\theta, z_n)$ behaves like $\nabla_{\theta} \ell_n(\theta, z_n)$ were normal random variables: $\nabla_{\theta} \ell_n(\theta, z_n) \sim N(g(\theta), C(\theta))$
- ◆ $\Rightarrow \hat{g}_s(\theta, z) - g(\theta) = \frac{1}{s} \sum_{n \in \mathcal{M}} (\nabla_{\theta} \ell_n(\theta, z_n) - g(\theta)) \sim N\left(0, \frac{1}{s} C(\theta)\right)$
- ◆ $\Rightarrow$ defining $\Delta g(\theta, z) = \sqrt{s}(\hat{g}_s(\theta, z) - g(\theta))$ implies $\Delta g(\theta, z) \sim N(0, C(\theta))$

Example: quadratic loss

- ◆ $\ell_n(\theta, z_n) = \frac{1}{2} \|z_n - \theta\|^2$, $\mathcal{L}(\theta) = \mathcal{L}(\theta, z) = \frac{1}{2N} \sum_{n=1}^N \|z_n - \theta\|^2$
- ◆ $\nabla_{\theta} \mathcal{L}(\theta, z) = \bar{z} - \theta$, with $\bar{z} = \sum_{n=1}^N z_n$, thus $\arg \min_{\theta} \mathcal{L}(\theta, z) = \bar{z}$
 - the reparametrization $\theta_{\text{new}} = \theta - \bar{z}$ leads to $\arg \min_{\theta_{\text{new}}} \mathcal{L}(\theta_{\text{new}}, z) = 0$
- ◆ $\hat{\mathcal{L}}_s(\theta) = \frac{1}{2s} \sum_{n \in \mathcal{M}} \|z - \theta\|^2$, $\hat{g}_s(\theta) = \nabla_{\theta} \hat{\mathcal{L}}_s(\theta) = \frac{1}{s} \sum_{n \in \mathcal{M}} (z_n - \theta)$
- ◆ The covariance of z_n is $\frac{1}{s} C(\theta) = \frac{1}{s} \mathbb{E}[(z_n - \bar{z})(z_n - \bar{z})^{\top}]$

Assumption 2

- ◆ In a *neighborhood* Θ of an *minimal* $\mathcal{L}(\theta)$ is $C(\theta)$ *constant* and *positive-definite*: $\exists$ a positive-definite $C \forall \theta \in \Theta: C(\theta) = C$
- ◆ Positive definiteness of $C \Rightarrow \exists$ a regular matrix $B: C = BB^\top$
- ◆ Defining $\Delta\theta(t) = \theta(t+1) - \theta(t)$, $\Delta w(\theta, z) = -B^{-1}\Delta g(\theta, z)$ implies:
 1. $\Delta\theta(t) = -\epsilon g(\theta(t)) - \frac{\epsilon}{\sqrt{s}}\Delta g(\theta(t), z) = -\epsilon g(\theta(t)) + \frac{\epsilon}{\sqrt{s}}B\Delta w(\theta(t), z)$
 2. $\Delta w(\theta, x) \sim N(0, -B^{-1}C(\theta)(-B^\top)^{-1}) = N(0, B^{-1}BB^\top(B^\top)^{-1}) = N(0, I)$

Assumption 3

- ◆ The equation $\Delta\theta(t) = -\epsilon g(\theta(t)) + \frac{\epsilon}{\sqrt{s}} B \Delta w(\theta(t), z)$, $\Delta w(\theta, z) \sim N(0, I)$, which is a finite-difference equation for $\Delta\theta(t) = \theta(t+1) - \theta(t)$, *is replaceable with* a differential equation: $\frac{d\theta}{dt} = -\epsilon g(\theta) + \frac{\epsilon}{\sqrt{s}} B \frac{dw}{dt}$
- ◆ The equation is particularly simple for $\mathcal{L}(\theta) = \frac{1}{2} \theta^\top A \theta$
 $\Rightarrow g(\theta) = A\theta \Rightarrow \frac{d\theta}{dt} = -\epsilon A\theta + \frac{\epsilon}{\sqrt{s}} B \frac{dw}{dt}$ – Ornstein-Uhlenbeck

Assumption 4

- ◆ The *loss* function is *on Θ quadratic*
- ◆ If for the original parameters $\mathcal{L}(\theta') = \theta'^{\top} A' \theta' + b \theta' + c'$, then the transformation $\theta = \theta' + \frac{1}{2} A'^{-1} b, A = 2A', c = c' - \frac{1}{4} b^{\top} A'^{-1} A'^{-1} b$ yields $\mathcal{L}(\theta) = \frac{1}{2} \theta^{\top} A \theta + c \implies g(\theta) = A \theta \implies \frac{d\theta}{dt} = -\epsilon A \theta + \frac{\epsilon}{\sqrt{s}} B \frac{dw}{dt}$
 - the solution of this differential equation is a random process called *Ornstein-Uhlenbeck process*

Example: quadratic loss

- ◆ $\ell_n(\theta, z_n) = \frac{1}{2} \|z_n - \theta\|^2, \mathcal{L}(\theta) = \mathcal{L}(\theta, z) = \frac{1}{2N} \sum_{n=1}^N \|z_n - \theta\|^2$
- ◆ $\nabla_{\theta} \mathcal{L}(\theta, z) = \bar{z} - \theta$, with $\bar{z} = \sum_{n=1}^N z_n$, thus $\arg \min_{\theta} \mathcal{L}(\theta, z) = \bar{z}$
 - the reparametrization $\theta_{\text{new}} = \theta - \bar{z}$ leads to $\arg \min_{\theta_{\text{new}}} \mathcal{L}(\theta_{\text{new}}, z) = 0$
- ◆ $\hat{\mathcal{L}}_s(\theta) = \frac{1}{2s} \sum_{n \in \mathcal{M}} \|z_n - \theta\|^2, \hat{g}_s(\theta) = \nabla_{\theta} \hat{\mathcal{L}}_s(\theta) = \frac{1}{s} \sum_{n \in \mathcal{M}} (z_n - \theta)$
- ◆ The covariance of x_n is $\frac{1}{s} C(\theta) = \frac{1}{s} \mathbb{E}[(z_n - \bar{z})(z_n - \bar{z})^{\top}]$
- ◆ The Hessian of $\mathcal{L}(\theta)$ is the identity matrix, $A = I$

Comments to the assumptions

- ◆ Valid in a neighborhood Θ of a $\mathcal{L}(\theta)$ local minimum
- ◆ About the Ornstein-Uhlenbeck process is known:
 1. θ for every t fulfills: $\theta(t) = e^{-At}\theta(0) + \sqrt{\frac{\epsilon}{s}} \int_0^t e^{-A(t-t')} B dW(t')$
 2. it has a *stationary distribution* $q(\theta)$, which is *Gaussian*
 3. for *its variance* $\Sigma = \mathbb{E}[\theta\theta^\top]$, matrices A, B, C from assumptions 2, 4, minibatch size s and learning rate ϵ : $\Sigma A + A \Sigma = \frac{\epsilon}{s} B B^\top = \frac{\epsilon}{s} C$

Encountered SGD strategies

1. *θ -maximize* the estimated *posterior probability* $p(\theta|x) = \frac{p(\theta,z)}{p(z)}$
 2. Tune the SGD so that the Ornstein-Uhlenbeck *stationary distribution* $q(\theta)$ possibly well *approximates* $p(\theta|z)$
- ◆ Both consider $\mathcal{L}(\theta) = \frac{1}{N} \sum_{n \in \hat{N}} \ell_n(\theta), \ell_n(\theta) = -\log p(z_n|\theta) - \frac{1}{N} \log p(\theta)$
- $\Rightarrow p(\theta|z) = \frac{p(z|\theta)p(\theta)}{p(z)} = \frac{\prod_{n=1}^N p(z_n|\theta)p(\theta)}{p(z)} = e^{-\log p(z)} e^{-N\mathcal{L}(\theta)}$
 - assumption 4 $\Rightarrow$ the posterior distribution of θ is $N\left(0, \frac{1}{N} A^{-1}\right)$

Approximation of the posterior

- ◆ Learning rate ϵ and minibatch size s are chosen so that they minimize the Kullback Leibler divergence $KL(q||p(\cdot |z))$ from the posterior distribution $p(\cdot |z)$ to the stationary distribution q
- ◆ The *learning rate* ϵ^* and *minibatch size* s^* corresponding to $\min KL(q||p(\cdot |z))$ are known to be related through

$$\epsilon^* = \frac{s^*}{N} \frac{D}{\text{Tr}(C)} = \frac{s^*}{N} \frac{D}{\text{Tr}(BB^\top)}, \text{ where } D \text{ denotes the dimension of } \theta$$

Componentwise approximation

- ◆ SGD with a componentwise descent: $\theta(t+1) = \theta(t) - G \hat{g}_s(\theta(t))$
 - G – a diagonal matrix, the original SGD corresponds to $G = \epsilon I$
- ◆ The diagonal matrix G^* and minibatch size s^* corresponding to $\min KL(q||p(\cdot | z))$ are known to be related through

$$G_{k,k}^* = \frac{2s^*}{NC_{k,k}} = \frac{2s^*}{N(BB^\top)_{k,k}} \text{ for each component } k = 1, \dots, D$$

Preconditioned approximation

- ◆ SGD with a preconditioning matrix H : $\theta(t+1) = \theta(t) - H\hat{g}_s(\theta(t))$
 - H is *positive definite*, the original SGD corresponds to $H = \epsilon I$
- ◆ The matrix H^* and minibatch size s^* corresponding to $\min KL(q||p(\cdot|z))$ are known to yield $\min KL(q||p(\cdot|z)) = 0$ and to be related through $H^* = \frac{2s^*}{N} C^{-1} = \frac{2s^*}{N} (BB^\top)^{-1}$